

ASCL HANDS-ON TRAINING · DAY 1 OF 4

ROS 2 & LIMO Cobot Training

From Robot Concepts to Real-World Action

LAB	Automated Systems & Computing Lab (ASCL)
LAB LEADER	Prof. Ahmad Taher Azar

Dr Saim Ahmed
Trainer 1 · Day 1 & Day 3

Eng. Zeeshan Haider
Trainer 2 · Day 2 & Day 4



ASCL
Automated Systems & Computing Lab

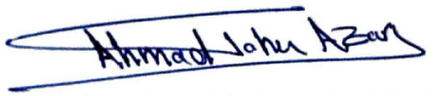
جامعة الامير سلطان
PRINCE SULTAN
UNIVERSITY



ACKNOWLEDGMENTS

This Training is supported by a research grant funded by the Research, Development, and Innovation Authority (RDIA), Kingdom of Saudi Arabia, with grant number 13382-psu-2023- PSNU-R-3-1-EI.

Prof. Ahmad Taher Azar



Full Professor, Leader of Automated Systems & Computing Lab (ASCL)

TRAINING AGENDA · 4 DAYS

Table of Contents

TRAINING 1 · DAYS 1–2 · FUNDAMENTALS

01 **Day 1 · What is ROS & ROS2?**

- Sense → Think → Act & the group-chat model
- Core concepts: nodes, topics, messages, services, actions
- turtlesim hands-on lab

02 **Day 2 · ROS2 Hands-On: CLI & First Nodes**

- The `ros2` CLI: node · topic · service · param
- Workspaces & packages
- First publisher & subscriber in Python

TRAINING 2 · DAYS 3–4 · ON THE ROBOT

03 **Day 3 · Meet the LIMO Cobot: Hardware & Bring-Up**

- Hardware, motion modes & sensors
- Connect & bring-up: NoMachine + chassis driver
- Drive & sense

04 **Day 4 · Applications: SLAM, Navigation & the Arm**

- SLAM mapping with Cartographer
- Navigation with Nav2
- myCobot 280 arm control

ROADMAP

Today's plan

01
What is ROS?
The sense→think→act loop and the "group chat" that connects a robot's parts. 15 min

02
Core Concepts
Nodes, topics, messages, services, actions, parameters — with an analogy each. 10 min

03
turtlesim Lab
Start a real ROS2 system and drive a turtle with your keyboard. 30 min

Hands-on time is sacred — we never cut the demo. Every command is copy-paste ready.

BY THE END OF TODAY

You will be able to…

- Explain **what ROS is** in one sentence — no jargon allowed
- Map a real robot part to each step of **sense** → **think** → **act**
- Define the **6 core ideas** (node, topic, message, service, action, parameter)
- Start two ROS2 programs and watch them talk over a **topic**
- Inspect a running system with `ros2 node list`, `ros2 topic echo`, and `rqt_graph`

01

What is ROS?

And why ROS2?

Every robot does the same three things — and they all need to talk to each other.

THE LOOP EVERY ROBOT RUNS

Sense → Think → Act

👁️ Sense

Collect data about the world.

LIMO: LiDAR spins, measures distances → `/scan`

🧠 Think

Decide what to do next.

"Wall 0.5 m ahead → turn left"

🦾 Act

Make the motors move.

Publish a speed command → `/cmd_vel`

Each part is its **own little program** — the LiDAR driver, the brain, the motor driver. **How do they talk to each other?** That's the problem ROS solves.

THE ANSWER

ROS - the robot's messaging system

ROS (Robot Operating System) is **not an operating system** like Windows or Linux. It's **middleware** — a messaging framework that lets a robot's programs talk to each other.

It runs **on top of** an OS (Ubuntu), and lets each robot program find and message the others — without knowing where they live.

THE ONE ANALOGY THAT UNLOCKS EVERYTHING

The group-chat robot 🧠

📡 LiDAR person

Posts every second: **"I see a wall at 0.5 m."**

🧠 Brain person

Reads the wall posts, thinks, posts: **"Turn left."**

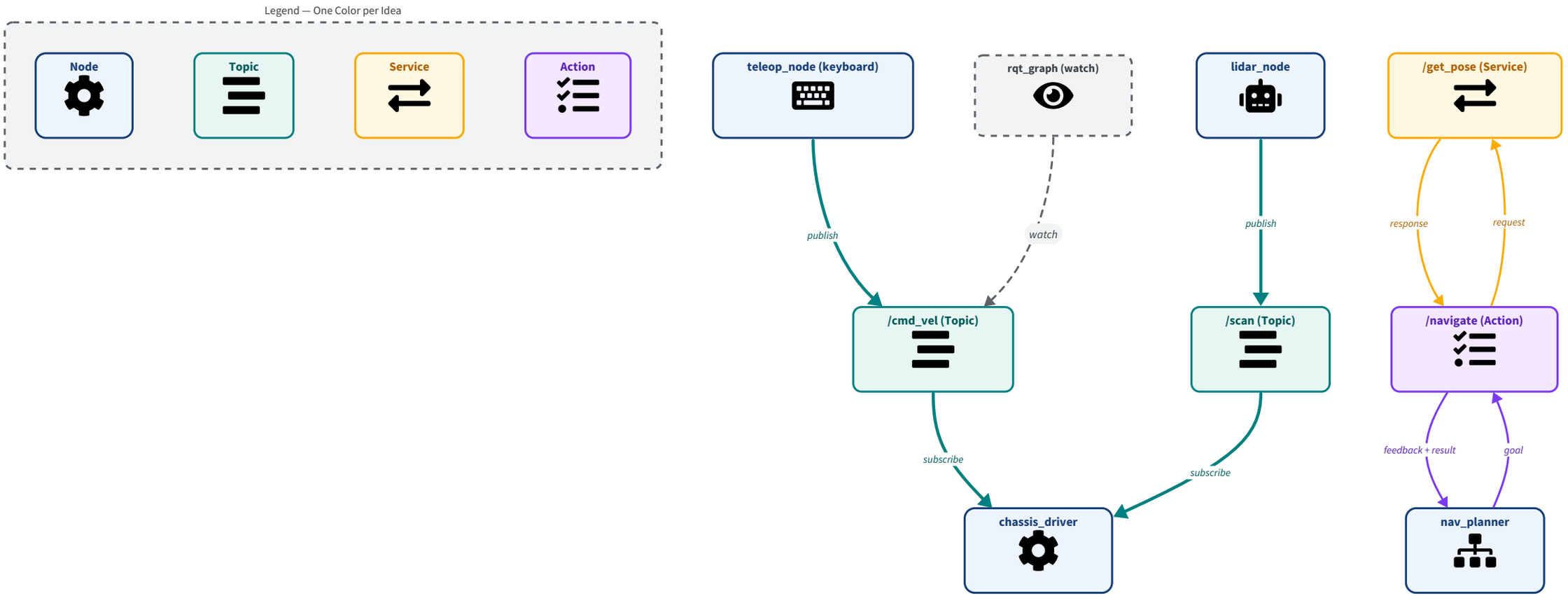
🦾 Motor person

Reads the turn posts, moves the wheels.

> Nobody is the boss of the chat — **anyone can post, anyone can read.** That's the ROS model: publish / subscribe.

THE PICTURE BEHIND THE ANALOGY

The ROS Graph



Blue boxes = nodes (programs) · teal boxes = topics (chatrooms) · the dashed box is `rqt_graph`

A FAIR QUESTION

Why not one big program?

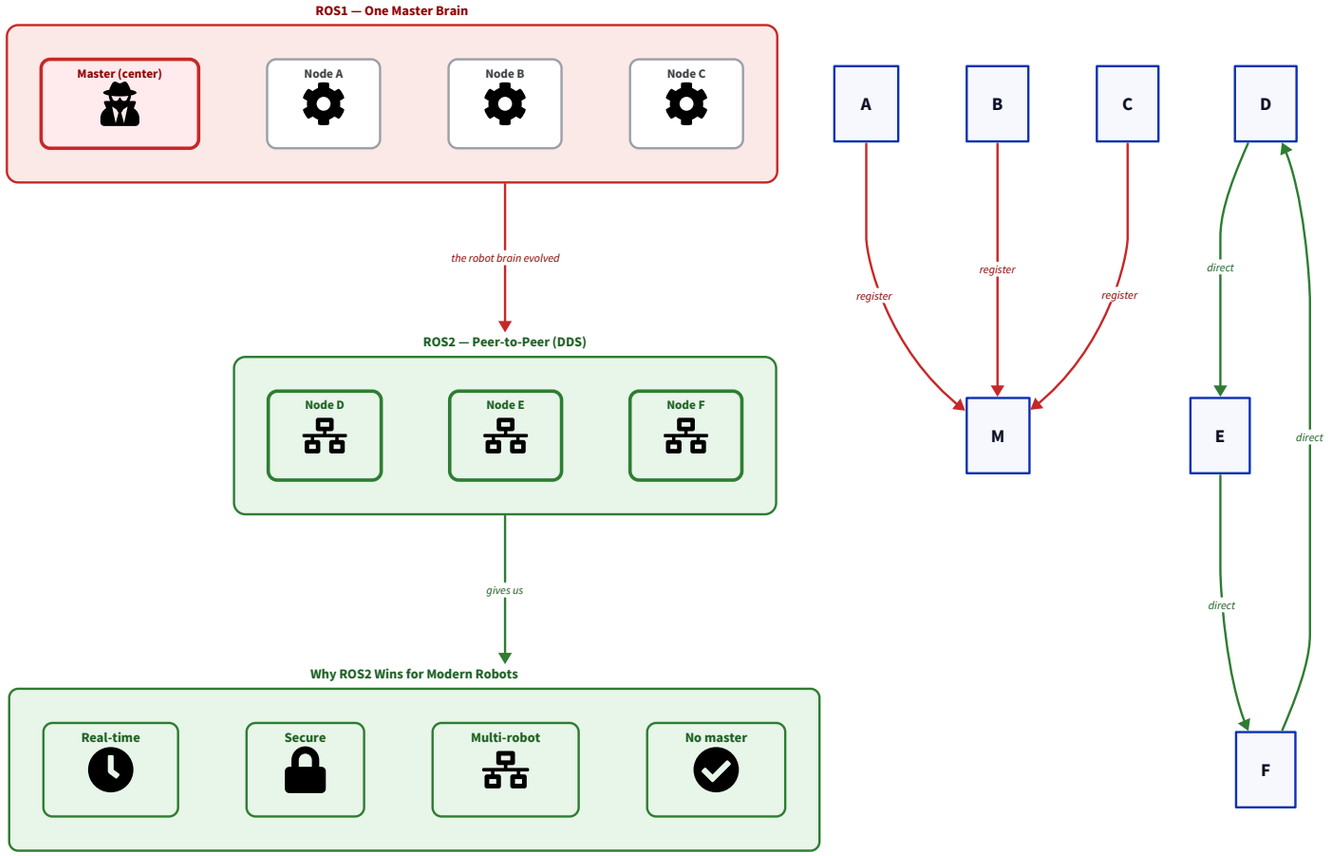
- Robots are built by **many teams**: LiDAR vendor, motor vendor, your lab's "brain".
- ROS lets pieces **plug together like LEGO**.
- Swap the LiDAR? The new driver publishes the **same** `/scan` topic — the brain never notices.

The killer feature: modularity — replace one piece without touching the rest.

HISTORY IN 30 SECONDS

ROS1 vs ROS2 — why ROS2?

ROS1 (2007) had one weakness: a **central "Master"**. If it died, the robot went silent.



ROS1 (left, red) funnels everything through one Master · ROS2 (right, green) lets nodes talk **directly**.

02

Core Concepts

Nodes, topics, messages, services, actions, parameters — the 6 ideas every robot is built from.

INSTALLATION

Installation- Ubuntu & ROS2

Run the Commands in PowerShell (run as Administrator).

```
wsl --install -d Ubuntu-24.04
```

```
wsl --list --verbose
```

Try it

https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html

TRY EXAMPLE

Talker-Listner

- source /opt/ros/jazzy/setup.bash
- ros2 run demo_nodes_cpp talker
- ros2 run demo_nodes_py listener

Try it

```
https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html
```

SIX IDEAS, SIX ANALOGIES

The vocabulary of ROS2

1 · Node
a person in the chat

2 · Topic
a TV channel

3 · Message
the post content

4 · Service
a phone call

5 · Action
food delivery

6 · Parameter
the settings menu

IDEA 1 OF 6

Node — "a person in the chat"

A **node** is one running program with one job.

- LiDAR node: read the laser, publish distances.
- Motor node: read speed commands, spin wheels.
- A robot = **many nodes working together**.

Try it

```
ros2 node list   # prints every running node
```

IDEAS 2 & 3 OF 6

Topic & Message

📺 Topic — a TV channel

A named channel carrying a **continuous stream** of data.

- Publishers **broadcast**.
- Subscribers **tune in**.
- Many-to-many, by name.

📄 Message — the content

The **structured data** on a topic.

The famous one — `Twist`, the speed command:

`geometry_msgs/msg/Twist`

```
linear:  {x: 0.1, y: 0.0, z: 0.0}   # forward 0.1 m/s
angular: {x: 0.0, y: 0.0, z: 0.5}   # turning 0.5 rad/s
```

Every topic has a fixed message type — `/scan` always carries `LaserScan`. That's the contract.

IDEA 4 OF 6

Service — "calling a helpline"

A **service** is a one-shot **request** → **response** exchange. You call, it answers once, done.

Example

`/get_pose` — "Where am I?" → the robot answers with coordinates.

Best for

One-shot questions. **Not** continuous data — that's a topic's job.

📺 **Topic** = a TV channel (continuous, many viewers) · ☎️ **Service** = a phone call (one question, one answer).

IDEA 5 OF 6

Action — "ordering food delivery"

An **action** is a **long-running task with progress updates** — and you can cancel it.

Action part	Food-delivery analogy	ROS2 part
Goal	"Deliver pizza to my house"	The target ("navigate to X")
Feedback	"Preparing... on the way..."	Progress while moving
Result	"Pizza delivered"	The final outcome

Navigation takes seconds and needs progress + cancel → that's why it's an **action**, not a service or topic.

IDEA 6 OF 6

Parameter — "the settings menu"

A **parameter** is a node's configuration value — tune it without rewriting code.

- The teleop node's speed multiplier.
- The robot's max velocity.

Set a setting live

```
ros2 param set <node> <param> <value>
```

PUTTING IT TOGETHER

One picture, six ideas

Color	Concept	Analogy	Data direction
🔵 Blue box	**Node**	Person in the chat	—
🟢 Teal box	**Topic**	TV channel	continuous stream
📄 inside topic	**Message**	Post content	structured data
🟡 Amber box	**Service**	Phone call	request → response
🟣 Purple box	**Action**	Food delivery	goal → feedback → result
⚙️ on node	**Parameter**	Settings menu	config value

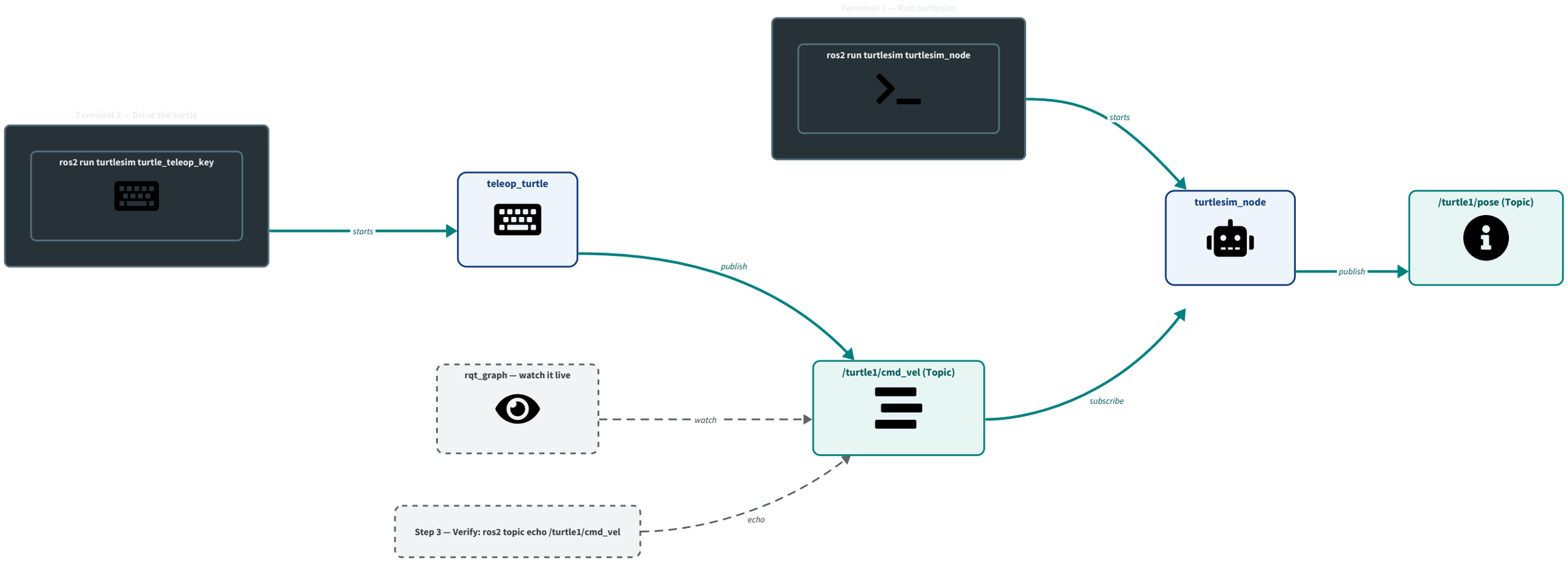
03

The turtlesim Lab

Your first live ROS2 system — a turtle you drive with your keyboard.

THE GOAL

Two nodes, one topic



Two terminals start two nodes · teleop publishes `/turtle1/cmd_vel` · turtlesim subscribes

STEP 0

Open 3 terminals

Every terminal needs ROS2 sourced first:

All terminals

```
source /opt/ros/jazzy/setup.bash
```

> "command not found: ros2"? You skipped the source line — ask an instructor.

STEP 1 · TERMINAL 1

Start the "robot"

Terminal 1 — the simulator node

```
ros2 run turtlesim turtlesim_node
```

A window pops up with a turtle. It's publishing its **pose** on `/turtle1/pose`, and waiting for speed commands on `/turtle1/cmd_vel`.

STEP 2 · TERMINAL 2

Start the "driver"

Terminal 2 — the keyboard node

```
ros2 run turtlesim turtle_teleop_key
```

Press the **arrow keys** (↑ ↓ ← →). The turtle moves! 🎉

> **Say it out loud:** the teleop node publishes a `Twist` on `/turtle1/cmd_vel`; the turtlesim node subscribes and moves. **Two nodes, one topic, no central boss.**

STEP 3 · TERMINAL 3

Become the detective 🔍

Who's running?

nodes

```
ros2 node list
```

→ `/turtlesim` and `teleop_turtle`

What's flowing?

topics

```
ros2 topic list
```

→ `/turtle1/cmd_vel` · `/turtle1/pose`

The topic connecting our two nodes is `/turtle1/cmd_vel`.

What is inside nodes?

nodes

```
ros2 topic info
```

→ `/turtle1/cmd_vel` · `/turtle1/pose`

See the message

info

```
ros2 topic info /turtle1/cmd_vel
```

We can see the message over topic.

STEP 3 · TERMINAL 3 (CONTINUED)

Watch the data & see the graph

Watch live data

echo

```
ros2 topic echo /turtle1/cmd_vel
```

Press arrow keys → watch `linear.x` and `angular.z` change. `Ctrl+C` to stop.

Publish parameters

graph

```
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.5, y: 0.5, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.1}}"
```

See the whole system

graph

```
rqt_graph
```

Two ellipses (nodes) + one rectangle (topic). **The diagram — now real.**

STEP 4 · BONUS

Teleport a turtle with a service

Remember: services = phone calls. Let's make one:

Terminal 3 — call /spawn

```
ros2 service call /spawn turtlesim/srv/Spawn \
  "{x: 2, y: 2, theta: 0.2, name: 'turtle2'}"
```

Terminal 4 — Teleop-key

```
ros2 run turtlesim turtle_teleop_key --ros-args --remap turtle1/cmd_vel:=turtle2/cmd_vel --remap turtle1/rotate_absolute:=turtle2/rotate_absolute
```

A **second turtle** appears. You made a request; the node answered with a response — that's a service in action.

WHAT JUST HAPPENED?

Lab recap

What you did	Concept it demonstrates
Started 2 programs	**Nodes** (one job each)
Arrow keys → turtle moved	**Topic** `/turtle1/cmd_vel` carrying **Twist messages**
`ros2 node list` / `topic list`	Introspection — see what's running
`ros2 topic echo`	Subscribe — watch live data
`rqt_graph`	The **ROS Graph** — one picture of the system
Spawned turtle2	**Service** — request/response

THE BIG TAKEAWAY

They connected by name, not by address

You never told the teleop node **"go talk to the turtle."** They found each other **through the topic name**.

Programs connect **by name**, not by address. That's the magic of ROS.



Quick Check

Five minutes. Questions — Answers revealed on each slide.

Q1 · What is ROS, in plain words?

A) An operating system like Windows, but for robots

B) A messaging system that lets a robot's programs talk to each other

C) A programming language for robots

D) A type of robot hardware

▸ **Show answer**

Q2 · What is a "topic" in ROS2?

A) A single program running on the robot

B) The robot's central brain

C) A named channel carrying a continuous stream of data between nodes

D) A setting stored inside a node

▸ **Show answer**

Q3 · Topic vs Service — which is correct?

A) A topic is one request/response; a service is a continuous stream

B) A topic is a continuous stream; a service is one request → one response

C) They are the same thing with different names

D) A service is only for 3D data

▸ **Show answer**

Q4 · Why is navigating to a goal an ACTION, not a service?

A) Actions are faster than services

B) Actions only work for moving robots

C) Navigation is long-running and needs progress feedback + cancel

D) Services cannot send coordinates

▸ **Show answer**

Q5 · Which command shows the live speed commands between two nodes?

A) `ros2 node list`

B) `rqt_graph`

C) `ros2 topic echo /turtle1/cmd_vel`

D) `ros2 service call /spawn`

▸ **Show answer**

Q6 · In the LIMO, which topic does the LiDAR publish to?

A) `/cmd_vel`

B) `/scan`

C) `/get_pose`

D) `/navigate`

▸ **Show answer**

DAY 1 COMPLETE ✅

Tomorrow: you write code.

Day 2 turns you into a **detective** — interrogate a running system with the `ros2` CLI, then write your **first publisher & subscriber** in Python.